You can add durations and subtract durations from instances of `std::chrono::time_point<>` to produce new time points, so `std::chrono::high_resolution_clock::now() + std::chrono::nanoseconds(500)` will give you a time 500 nanoseconds in the future. This is good for calculating an absolute timeout when you know the maximum duration of a block of code, but there are multiple calls to waiting functions within it or nonwaiting functions that precede a waiting function but take up some of the time budget.

You can also subtract one time point from another that shares the same clock. The result is a duration specifying the length of time between the two time points. This is useful for timing blocks of code, for example:

```
auto start=std::chrono::high_resolution_clock::now();
do_something();
auto stop=std::chrono::high_resolution_clock::now();
std::cout<<"do_something() took "
  <<std::chrono::duration<double,std::chrono::seconds>(stop-start).count()
  <<" seconds"<<std::endl;
```

The clock parameter of a `std::chrono::time_point<>` instance does more than just specify the epoch, though. When you pass the time point to a wait function that takes an absolute timeout, the clock parameter of the time point is used to measure the time. This has important consequences when the clock is changed, because the wait tracks the clock change and won't return until the clock's `now()` function returns a value later than the specified timeout. If the clock is adjusted forward, this may reduce the total length of the wait (as measured by a steady clock), and if it's adjusted backward, this may increase the total length of the wait.

As you may expect, time points are used with the `_until` variants of the wait functions. The typical use case is as an offset from *some-clock*`::now()` at a fixed point in the program, although time points associated with the system clock can be obtained by converting from a `time_t` using the `std::chrono::system_clock::to_time_point()` static member function for scheduling operations at a user-visible time. For example, if you have a maximum of 500 milliseconds to wait for an event associated with a condition variable, you might do something like in the following listing.

Listing 4.11. Waiting for a condition variable with a timeout

```
#include <condition_variable>
#include <mutex>
#include <chrono>
std::condition_variable cv;
bool done;
std::mutex m;

bool wait_loop()
{
     auto const timeout= std::chrono::steady_clock::now()+
         std::chrono::milliseconds(500);
     std::unique_lock<std::mutex> lk(m);
     while(!done)
     {
        if(cv.wait_until(lk,timeout)==std::cv_status::timeout)
            break;
     }
```